

KV cache dei modelli LLM: tabella e formula di calcolo

Oct 2, 2026 · @Alvise Dorigo

Tabella: KV cache fp16 a 128K

A 128K token e 16 bit la KV cache va da 0,75 GiB (Nemotron-3.5-Lightning) a 40 GiB (Llama-3.3-70B). Conta quanti layer tengono K e V su tutto il contesto, non la dimensione del modello.

Modello	Ctx max	Layer con KV su tutto il contesto	KV per token	KV @128K	con MTP	Note
Nemotron-3.5-Lightning-30B-A3B	256K (fino a 1M)	6 di 52 (resto Mamba-2/MoE)	6 KiB	0,75 GiB	0,88 GiB	+ stato Mamba ~47 MiB
Nemotron-3-Super-120B-A12B	256K (fino a 1M)	8 di 88 (resto Mamba-2/MoE)	8 KiB	1,0 GiB	1,13 GiB	+ stato Mamba ~162 MiB
Qwen3.6-35B-A3B-MTP	256K	10 di 40 (resto DeltaNet)	20 KiB	2,5 GiB	2,75 GiB	+ stato DeltaNet ~61 MiB
Ornith-1.5-35B-A3B	256K (~1M con YaRN)	10 di 40 (resto DeltaNet)	20 KiB	2,5 GiB	2,75 GiB	architettura come Qwen3.6; + stato DeltaNet ~61 MiB
Gemma-4-26B-A4B	256K	5 di 30 (resto finestra 1.024)	20 KiB	2,7 GiB	—	incl. 200 MiB di finestra; con <code>--swa-full</code> 27,5 GiB
Qwen3-Coder-Next (80B-A3B)	256K	12 di 48 (resto DeltaNet)	24 KiB	3,0 GiB	3,25 GiB ²	+ stato DeltaNet ~74 MiB
GPT-OSS-120b	128K	18 di 36 (resto finestra 128)	36 KiB	4,5 GiB	—	con <code>--swa-full</code> 9 GiB
GLM-4.7-Flash	~198K	47 di 47 (MLA compressa)	52,9 KiB	6,6 GiB	6,75 GiB	MLA puro; llama.cpp (log di gennaio) ~12,5 GiB; senza MLA 117,5 GiB
Llama-4-Scout-17B-16E	10M	12 di 48 (resto blocchi 8.192)	48 KiB	7,1 GiB	—	incl. 1,1 GiB di blocchi; con <code>--swa-full</code> 24 GiB
Qwen3.8-27B	256K	16 di 64 (resto DeltaNet)	64 KiB	8,0 GiB	8,5 GiB ¹	+ stato DeltaNet ~147 MiB

Modello	Ctx max	Layer con KV su tutto il contesto	KV per token	KV @128K	con MTP	Note
Gemma-4-31B	256K	10 di 60 (resto finestra 1.024)	80 KiB	10,8 GiB	—	incl. 800 MiB di finestra; con <code>--swa-full</code> 110 GiB
Llama-3.3-70B	128K	80 di 80	320 KiB	40 GiB	—	attenzione piena classica

- Valori per singola sequenza, con K e V a 16 bit; 128K = 131.072 token; GiB = 2³⁰ byte. Righe ordinate per KV crescente.
- Stato ricorrente (Mamba/DeltaNet): in float32 come da config (per Qwen3-Coder-Next è un'assunzione). È fisso, non cresce col contesto ed è escluso dal totale.
- ¹ MTP della Qwen3.8 assunto uguale a quello della 3.6 (1 layer ad attenzione piena), non verificato.
- ² Vale solo se il checkpoint include la testa MTP: non verificato per Qwen3-Coder-Next.
- Gemma 4, K=V: sfruttando il riuso delle chiavi come valori, la parte globale scenderebbe del 37,5% (26B → 1,8 GiB; 31B → 7,0 GiB). llama.cpp però alloca comunque la V a parte.

Formula generale

Per l'attenzione standard la KV per token si ottiene moltiplicando 2 (K e V) per i layer con KV, le teste KV, la dimensione per testa e i byte per elemento. Il totale è quel valore per il numero di token, più l'eventuale parte fissa (finestre, blocchi).

$$\mathrm{KV}_{\mathrm{token}} = 2 \cdot L_{\mathrm{KV}} \cdot n_{\mathrm{kv}} \cdot d_{\mathrm{head}} \cdot b$$

$$\mathrm{KV}_{\mathrm{tot}} = \mathrm{KV}_{\mathrm{token}} \cdot N_{\mathrm{ctx}} + \mathrm{parte\ fissa}$$

Simbolo	Significato	Dove si legge nel config.json
2	un vettore K e uno V per token	costante; con MLA vale 1 (vedi casi speciali)
L_KV	layer che tengono K e V su tutto il contesto	<code>num_hidden_layers</code> , ridotto contando le voci di attenzione piena in <code>layer_types</code> (oppure <code>num_hidden_layers / full_attention_interval</code>)

Simbolo	Significato	Dove si legge nel config.json
n_kv	teste KV (non le teste di query)	num_key_value_heads ; Gemma 4 globali: num_global_key_value_heads
d_head	dimensione di ogni testa	head_dim ; Gemma 4 globali: global_head_dim ; se manca: hidden_size / num_attention_heads
b	byte per elemento	2 per fp16/bf16; 1 per fp8; circa 1,06 per q8_0 e 0,56 per q4_0 di llama.cpp
N_ctx	token di contesto per sequenza	scelto da te; massimo nativo = max_position_embeddings

Scorciatoia: con KV per token espresso in KiB, a 128K (2^{17} token) il totale in GiB è KV per token / 8; a 256K è KV per token / 4. Esempio: 20 KiB per token → 2,5 GiB a 128K.

Il valore è per una sequenza. Con più richieste in parallelo la KV scala con i token totali in cache, lo stato ricorrente con il numero di sequenze.

Casi speciali

La formula base vale solo per i layer che vedono tutto il contesto. Layer ricorrenti, finestre, MLA e teste MTP cambiano il conteggio come segue.

Layer ricorrenti (DeltaNet, Mamba-2)

In L_KV entrano solo i layer di attenzione piena. Per Qwen3.5/3.6/3.8, Qwen3-Next e Ornith sono le voci `full_attention` in `layer_types`. Per Nemotron sono i simboli `*` in `hybrid_override_pattern` o le voci `attention` in `layers_block_type`.

Gli altri layer tengono uno stato fisso per sequenza, che non cresce col contesto:

$$S_{\{\mathrm{DeltaNet}\}} = L_{\{\mathrm{lin}\}} \cdot n_v \cdot d_k \cdot d_v \cdot b_s$$

Qui $n_v = \text{linear_num_value_heads}$, $d_k = \text{linear_key_head_dim}$, $d_v = \text{linear_value_head_dim}$. b_s vale 4 se `mamba_ssm_dtype` è float32.

$$S_{\{\mathrm{Mamba2}\}} = L_{\{\mathrm{mamba}\}} \cdot n_h \cdot d_h \cdot d_{\{\mathrm{state}\}} \cdot b_s$$

Qui $n_h = \text{mamba_num_heads}$, $d_h = \text{mamba_head_dim}$, $d_{\text{state}} = \text{ssm_state_size}$, b_s da $\text{mamba_ssm_cache_dtype}$. Lo stato della convoluzione aggiunge pochi MiB: $(\text{kernel} - 1) \times \text{canali} \times \text{byte}$. I canali sono $2 \times n_k \times d_k + n_v \times d_v$ per DeltaNet, $n_h \times d_h + 2 \times n_{\text{groups}} \times d_{\text{state}}$ per Mamba-2.

Finestra scorrevole e attenzione a blocchi

I layer locali tengono al massimo W token, quindi finiscono nella parte fissa. Si riconoscono da `sliding_attention` in `layer_types` (Gemma 4, GPT-OSS) o da `no_rope_layers = 1` in Llama 4, dove 0 indica un layer globale.

$$\text{fisso}_{\text{loc}} = 2 \cdot L_{\text{loc}} \cdot n_{\text{kv,loc}} \cdot d_{\text{loc}} \cdot b \cdot \min(W, N_{\text{ctx}})$$

W è `sliding_window` (Gemma 4: 1.024; GPT-OSS: 128) oppure `attention_chunk_size` (Llama 4: 8.192). In Gemma 4 i layer locali usano `num_key_value_heads` e `head_dim`, quelli globali `num_global_key_value_heads` e `global_head_dim`.

llama.cpp alloca per i layer locali la finestra più il micro-batch, per esempio $1.024 + 512$ celle. Con `--swa-full` li alloca su tutto il contesto: in quel caso vanno contati in L_{KV} .

MLA (GLM-4.7-Flash, famiglia DeepSeek)

Si riconosce dalla presenza di `kv_lora_rank`. Per ogni token e layer si salva un vettore compresso, non K e V per ogni testa:

$$\text{KV}_{\text{token}} = L \cdot (r_{\text{kv}} + d_{\text{rope}}) \cdot b$$

Qui $r_{\text{kv}} = \text{kv_lora_rank}$ e $d_{\text{rope}} = \text{qk_rope_head_dim}$ (GLM-4.7-Flash: $512 + 64$). Il risultato reale dipende dal motore:

- llama.cpp (log di gennaio 2026) salva anche una V da r_{kv} valori: $L \times (2 \times r_{\text{kv}} + d_{\text{rope}}) \times b$.
- Un motore senza MLA salva K e V completi: $L \times \text{num_attention_heads} \times (\text{qk_nope_head_dim} + \text{qk_rope_head_dim} + \text{v_head_dim}) \times b$.

Teste MTP

Ogni layer MTP con attenzione piena aggiunge 1 a `L_KV`. I campi sono `mtp_num_hidden_layers` (Qwen) e `num_nextn_predict_layers` (GLM, Nemotron); in Nemotron il tipo di layer è in `mtp_hybrid_override_pattern` o `mtp_layers_block_type`. Verifica che il checkpoint contenga i tensori `mtp.*`: molte quantizzazioni li perdono senza segnalarlo.

K = V (Gemma 4)

Con `attention_k_eq_v: true` i layer globali riusano le chiavi come valori. Il technical report dichiara -37,5% sulla parte globale, ma llama.cpp alloca comunque la V: per una stima prudente tieni il fattore 2.

Contesto oltre il nativo

Con YaRN (`rope_scaling`) puoi superare `max_position_embeddings`. La formula non cambia, ma la qualità oltre il contesto nativo non è garantita.

Esempio guidato

Due esempi coprono i casi più comuni: un ibrido lineare (Qwen3.6-35B-A3B) e un modello con finestra scorrevole (Gemma-4-31B).

Qwen3.6-35B-A3B (ibrido DeltaNet)

1. Apri il `config.json`: il modello è multimodale, quindi i parametri sono dentro `text_config`.
2. Conta le voci `full_attention` in `layer_types`: 10 su 40. Controllo: $40 / \text{full_attention_interval} (4) = 10$.
3. Leggi `num_key_value_heads` = 2 e `head_dim` = 256. Attenzione: `hidden_size / num_attention_heads` darebbe 128, un valore sbagliato.
4. KV per token = $2 \times 10 \times 2 \times 256 \times 2 \text{ byte} = 20.480 \text{ B} = 20 \text{ KiB}$.
5. A 128K: $20 \text{ KiB} \times 131.072 = 2,5 \text{ GiB}$ (scorciatoia: $20 / 8$).
6. MTP: `mtp_num_hidden_layers` = 1 aggiunge 2 KiB per token, per un totale di 2,75 GiB.
7. Stato fisso: $30 \text{ layer} \times 32 \times 128 \times 128 \times 4 \text{ byte} = 60 \text{ MiB}$, più 1-3 MiB di convoluzione.

Gemma-4-31B (finestra scorrevole)

1. In `text_config`, `layer_types` contiene 50 voci `sliding_attention` e 10 `full_attention`; `sliding_window` = 1.024.
2. Globali: $2 \times 10 \times 4 \text{ (num_global_key_value_heads)} \times 512 \text{ (global_head_dim)} \times 2 \text{ byte} = 80 \text{ KiB}$ per token, cioè 10 GiB a 128K.
3. Locali: $2 \times 50 \times 16 \text{ (num_key_value_heads)} \times 256 \text{ (head_dim)} \times 2 \text{ byte} \times 1.024 \text{ token} = 800 \text{ MiB}$ fissi.
4. Totale a 128K: $10 \text{ GiB} + 800 \text{ MiB} \approx 10,8 \text{ GiB}$.
5. Con `--swa-full` i locali coprono tutto il contesto: 100 GiB invece di 800 MiB, per un totale di 110 GiB.

Dove trovare i parametri e trappole comuni

Quasi sempre basta il `config.json` su Hugging Face. Il valore reale si verifica poi nel log di avvio del motore.

Fonti

- Hugging Face, scheda Files and versions → `config.json`. Nei modelli multimodali i campi stanno dentro `text_config`.
- Repo ad accesso riservato (per esempio Meta): le copie pubbliche di Unsloth o mlx-community riportano gli stessi parametri architetturali.
- File GGUF: gli stessi dati sono nei metadati, per esempio `<arch>.block_count`, `<arch>.attention.head_count_kv`, `<arch>.attention.key_length`, `<arch>.attention.sliding_window`.
- Verifica sul campo: llama.cpp all'avvio stampa la riga `llama_kv_cache: size = ...` con K e V separati; vLLM stampa quanti token entrano nella cache.

Trappole

- `num_attention_heads` non è il numero di teste KV: usa `num_key_value_heads`.
- Un `head_dim` esplicito vince su `hidden_size / num_attention_heads` (Qwen3.6: 256 invece di 128).
- $256K = 262.144 \text{ token}$ e $128K = 131.072$; $1 \text{ GiB} = 2^{30} \text{ byte} \approx 1,074 \text{ GB}$.
- Layer locali senza limite di finestra (per esempio `--swa-full`): la cache cresce di un ordine di grandezza (Gemma-4-31B: da 10,8 a 110 GiB a 128K).

- MLA: il valore cambia col motore fino a 18 volte (GLM-4.7-Flash a 128K: 6,6, 12,5 o 117,5 GiB).
- KV quantizzata: b scende a circa 1,06 (q8_0) o 0,56 (q4_0). Lo stato ricorrente segue invece il proprio tipo, in vLLM `--mamba-ssm-cache-dtype`.
- Teste MTP: contare solo se i tensori `mtp.*` sono presenti nel file che usi.